Approximation algorithms

Part VII Selected Topics

Approximation trades the cost of exact optimization for a proved bound. Covering, metric TSP, and subset-sum illustrate lower bounds, approximation errors, and bit costs. Constant ratios, logarithmic ratios, and FPTAS guarantees are distinct. Prerequisites are optimization, approximation ratios, and NP-hard problems.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Chapter map

The sections cover vertex cover, traveling salesperson, set cover, randomization and linear programming, and subset sum. The goals are to prove approximation ratios and distinguish randomized guarantees, LP rounding, PTAS, and FPTAS.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Objectives, feasibility, and denominators

Objectives are nonnegative. A minimization ρ-approximation returns a feasible solution on every input with $ALG \leq \rho OPT$. Maximization requires $ALG \geq OPT/\rho$, $\rho \geq 1$. When OPT=0, use the inequalities rather than 0/0. Minimization must return zero, while any nonnegative feasible maximization value satisfies its inequality.

A good result on one example is not an approximation guarantee. The bound must hold on every legal input, and time is measured in encoded input length. A minimization proof often finds $LB \leq OPT$ and proves $ALG \leq \rho LB$. Maximization can use $UB \geq OPT$ and prove $ALG \geq UB/\rho$.

A PTAS runs in polynomial input length N for every fixed $\epsilon > 0$. For example, $N^{1/\epsilon}$ can qualify. An FPTAS is polynomial jointly in N and the numerical value $1/\epsilon$, not merely its logarithm. Very small epsilon can still be costly.

Unweighted vertex cover

For a simple undirected graph, a cover touches every edge. Minimizing its size is NP-hard. This algorithm's guarantee is for unweighted size. With no edges, return the empty cover and OPT=0.

```
C=empty; matched[v]=false for every v
for each edge (u,v), in a fixed order:
    if not matched[u] and not matched[v]:
        add u,v to C
        matched[u]=matched[v]=true
return C
```

Equivalently, choose an uncovered edge, add both endpoints, and delete covered edges. Marking endpoints avoids repeatedly rebuilding the graph and takes $O(|V| + |E|)$ time and $O(|V|)$ workspace, excluding input storage.

A maximal matching is a lower bound

Let M be the selected edges. Marking both endpoints immediately ensures no selected edges share a vertex, so M is a matching. An unselected edge already had a marked endpoint when scanned, and is covered in the final C. Thus C is feasible.

The matching is maximal, meaning no edge can be added, but need not be maximum in cardinality. Every cover, including an optimum C*, must choose at least one endpoint for each M edge. Disjoint endpoints require distinct charged vertices, so

$$|C| = 2|M| \leq 2|C^*|.$$

The proof needs neither OPT nor a maximum matching. Arbitrary adjacent edges cannot be charged independently to obtain an |E| lower bound, since they may share one selected vertex. A matching prevents that double counting.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Ratio two and exact optimality

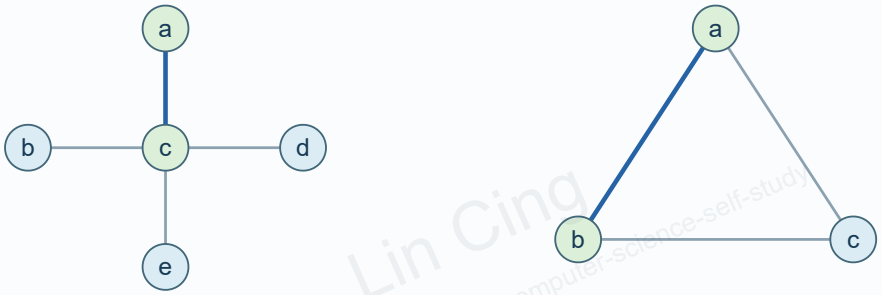
For a four-leaf star centered at c, choose edge (c,a). The output is {c,a}, while {c} is optimal, attaining ratio 2. For a triangle, choosing ab gives {a,b}. Every one-vertex choice misses its opposite edge, so OPT=2 and this output is optimal.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Ratio two and exact optimality (continued)

Matching endpoints: ratio 2 on a star, optimal on a triangle

Thick blue edges form the matching. Green vertices form the returned cover.



|M|=1, ALG=2, OPT=1 |M|=1, ALG=2, OPT=2

Weights invalidate this proof for weighted cost. A single edge with endpoint weights 1 and 100 gives algorithm cost 101 and optimum 1. Weighted vertex cover needs a weight-sensitive bound, such as LP rounding below.

Metric TSP and double tree

The main input is a complete undirected graph with n≥3, symmetric nonnegative costs, and triangle inequality $c(u, w) \leq c(u, v) + c(v, w)$. This is metric TSP. Distinct vertices may have zero distance, a pseudometric; the proofs still hold.

A tour visits every vertex once and returns to its start. Double tree computes an MST T, makes a full DFS walk from any root, takes vertices in first-visit order, and closes the resulting tour. Completeness ensures each shortcut edge exists. This construction cannot be applied directly to an arbitrary sparse graph.

Simple Prim on a full cost matrix takes $O(n^2)$, with linear DFS and output, for $O(n^2)$ word arithmetic operations and comparisons. Large rational weights require additional bit costs.

Each inequality in the factor-two proof

Deleting any edge from an optimal tour leaves a spanning tree. Nonnegativity means deletion does not increase cost, so MST minimality gives $c(T) \leq OPT$.

The full DFS walk W traverses each tree edge twice, giving $c(W) = 2c(T)$. It repeats vertices and is not yet a TSP tour.

Replace a walk section $u \to v_1 \to \cdots \to v_k \to w$, whose interior vertices were already visited, by $u \to w$. Repeated triangle inequality makes the shortcut no more expensive. Keeping first visits and final closure yields a feasible tour H with

$$c(H) \leq c(W) = 2c(T) \leq 2OPT.$$

The lower bound uses nonnegativity, shortcutting uses the metric inequality, and feasibility uses completeness. These assumptions have different roles.

Every walk edge and its actual cost

Use Manhattan distances for A(0,0),B(2,0),C(2,1),D(0,1). The coordinatewise absolute-value triangle inequalities imply the metric inequality.

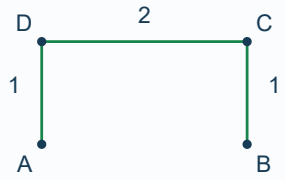
Take MST edges AD=1, DC=2, CB=1, totaling 4. The six complete-graph costs are 1,1,2,2,3,3. Every spanning tree uses three edges and therefore costs at least 1+1+2=4, proving this chain minimal.

Root at A. The full walk A,D,C,B,C,D,A costs 1+2+1+1+2+1=8. First-visit tour A,D,C,B,A costs 1+2+1+2=6. Any closed tour reaching both x-extremes has total x variation at least 4 and total y variation at least 2. Cost is at least 6, attained around the rectangle, so OPT=6. Walk cost 8 and tour cost 6 are different quantities.

Every walk edge and its actual cost (continued)

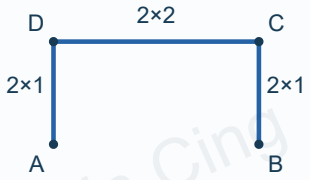
Same rectangle and Manhattan costs: MST 4 → walk 8 → tour 6

Minimum spanning tree



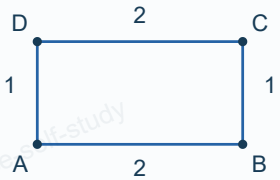
AD+DC+CB=1+2+1=4

Traverse each tree edge twice



A,D,C,B,C,D,A; cost 8

First visits, then return to A



A,D,C,B,A; cost 6

A(0,0), B(2,0), C(2,1), D(0,1). Axes have equal scale. Labels are Manhattan costs.

Christofides' three-halves guarantee

Under the same metric assumptions, let O be the odd-degree vertices of an MST. The degree sum is twice the edge count, so |O| is even. Find a minimum-weight perfect matching M on O and add it to T as a multigraph. Every odd degree becomes even, all other degrees remain even, and connectivity remains. An Euler closed walk exists; shortcut it to a tour.

Take an optimal tour and shortcut it to the vertices of O in visit order. Its closed cycle costs at most OPT. Alternating cycle edges form two perfect matchings. If |O|=2, view the cycle as two return edges. Their combined cost is at most OPT, so one costs at most OPT/2. Minimum matching therefore has $c(M) \leq OPT/2$. The final tour costs at most $c(T) + c(M) \leq 3OPT/2$.

This uses the subroutine theorem that general-graph minimum-weight perfect matching is polynomial-time solvable. An arbitrary greedy matching cannot replace the minimum matching in this proof. Euler traversal and shortcutting are polynomial too.

A constant ratio for nonmetric TSP would decide HC

Assume a deterministic polynomial-time algorithm A guarantees fixed ratio $\rho \geq 1$ on every nonnegative complete graph. Fix integer $R \geq \rho$. For an n-vertex HC input, give original edges weight 1 and nonedges weight $M = Rn + 1$.

With an HC, OPT=n, so A returns cost at most $\rho n \leq Rn$. Without an HC, every tour uses a nonedge, costing at least $M + n - 1 = Rn + n > Rn$. Comparing A's output to Rn decides HC in polynomial time, implying P=NP.

M has $O(\log n)$ binary bits, and the target graph has $O(n^2 \log n)$ bits. Large values conceal no superpolynomial output. These costs generally violate triangle inequality and do not contradict metric algorithms. The conclusion is conditional on P≠NP and applies to the stated deterministic guarantee. An expected-value randomized guarantee is not automatically a deterministic decider.

Weighted set cover

Let universe X have N elements and family $F = \{S_1, \ldots, S_M\}$, each a subset of X, with union X. Unweighted cover minimizes selected set count. Weighted cover has positive rational costs and minimizes total cost. Empty X returns the empty cover. If the union is insufficient, report infeasibility.

```
U=X; C=empty
while U is nonempty:
    choose S with |S intersect U|>0 minimizing w(S)/|S intersect U|
    C=C union {S}; U=U minus S
return C
```

For unit weights, this chooses maximum new coverage. Every round covers a new element, and previously chosen sets have zero new coverage. There are at most min(N,M) rounds. A direct incidence-matrix implementation computes counts in O(NM) per round and O(NM min(N,M)) operations overall, polynomial. Compare rational ratios exactly by cross multiplication.

Weighted set cover (continued)

Exact set cover is NP-hard. Map vertex cover to X=E, with one unit-cost set Sv of incident edges per vertex. At most k sets cover X iff at most k vertices cover the original edges. Incidence count is 2|E|, so size and construction are polynomial. Weighted set cover includes the unit-cost case. This does not claim every restricted family is hard.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Element charges yield a harmonic bound

When the greedy algorithm buys S of cost w covering k new elements, charge each new element w/k. Each element is charged exactly once, so total charge equals ALG.

Fix any candidate set A of size dA. Order its elements by first coverage as e1,...,edA, arbitrarily within one round. At the start of the round covering ej, A has at least dA-j+1 uncovered elements and is a valid alternative. Greedy's charge therefore satisfies

$$charge(e_j) \le \frac{w(A)}{d_A - j + 1}.$$

If other A elements were covered earlier in the same round, the number uncovered at the round's start is only larger, preserving the bound. Summing gives $\sum_{e \in A} charge(e) \le w(A) H_{d_A}$, where $H_d = \sum_{j=1}^{d} 1/j$ and H0=0.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Summing over an optimum cover

Let C* be optimal and d=max|Sj|. Every element belongs to at least one optimum set and has nonnegative charge. Thus

$$ALG = \sum_{e \in X} charge(e) \le \sum_{A \in C^*} \sum_{e \in A} charge(e) \le \sum_{A \in C^*} w(A) H_{|A|} \le H_d OPT.$$

Repeated elements on the right increase the upper bound, so the inequality direction is correct. For j≥2, $1/j \le \int_{j-1}^{j} dx/x$, proving $H_d \le 1 + \ln d$.

Weighted and unweighted cases share the proof but use different selection rules. Choosing only maximum new count with arbitrary weights does not minimize charge per element and has no guarantee from this argument.

Three greedy sets versus two optimal sets

Let X={1,2,3,4,5,6}, S1={1,2,3}, S2={4,5,6}, S3={1,2,4,5}, all cost 1. Break the later tie in favor of S1.

Round	U before selection	New counts S1,S2,S3	Choice	New charges
1	1,2,3,4,5,6	3,3,4	S3	1,2,4,5 each pay 1/4
2	3,6	1,1,0	S1	3 pays 1
3	6	0,1,0	S2	6 pays 1

ALG=3. Sets S1,S2 cover X, and no single set does, so OPT=2 and ratio=3/2. Charges sum to 3. Each optimum set receives total 3/2, at most $H_3 = 11/6$, as the general proof predicts.

Three greedy sets versus two optimal sets (continued)

S3 covers four first, but greedy needs three sets; S1+S2 need two

	1	2	3	4	5	6	
S1	1	1	1	0	0	0	Initially: 1,2,3,4,5,6
S2	0	0	0	1	1	1	After S3: 3,6
S3	1	1	0	1	1	0	After S1: 6
							After S2: ∅

charge(1,2,4,5)=1/4; charge(3)=charge(6)=1

Total charge 3 equals greedy cost. Optimal S1∪S2={1,…,6}; no single set covers X.

Randomized maximum three-CNF satisfaction

MAX-3-CNF-SAT maximizes satisfied clauses among m clauses. Require three distinct variables per clause and no complementary pair. Assign variables independently and fairly. Each clause has probability 1/8 of being entirely false and 7/8 of being true.

Exact polynomial optimization would decide three-CNF satisfiability by checking OPT=m, so exact optimization is NP-hard. It still admits the following approximation. Let Ij indicate satisfaction and Y=sum Ij. Clauses may share variables, so their indicators need not be independent. Linearity of expectation gives

$$E[Y] = \sum_j E[I_j] = 7m/8 \geq 7OPT/8.$$

An expected-objective 8/7 approximation does not promise that every run reaches 7OPT/8 or that $E[OPT/Y] \leq 8/7$. A single clause gives Y=0 with probability 1/8, preventing a finite expected-ratio statement. Repeated literals such as (a∨a∨a) have only 1/2 satisfaction probability. A clause containing a literal and its negation is always true and can be counted separately.

From expectation to a success probability

For m=0, any assignment gives Y=OPT=0 and succeeds with probability 1. For m>0, let $p = \Pr[Y \geq 3m/4]$. Since 0≤Y≤m and E[Y]=7m/8,

$$7m/8 \leq (1-p)3m/4 + pm,$$

so p≥1/2. Independently rerun r times and retain the best assignment. Failure to reach 3m/4 has probability at most 2^{-r}. This is a high-probability 4/3 threshold guarantee. It is not a high-probability 7/8 threshold guarantee.

For a deterministic 7/8 objective bound, fix variables by conditional expectation instead.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Conditional expectation never decreases

Given a partial assignment, a clause with a true literal has conditional satisfaction probability 1. A fully assigned false clause has probability 0. Otherwise, with r unassigned distinct variables, it has probability $1 - 2^{-r}$. Sum these probabilities to compute the conditional expectation exactly.

For the next variable x, the current expectation is the average of its values after fixing x=0 and x=1. Choose the larger branch. After all variables are fixed, expectation equals actual satisfaction count, at least $7m/8 \geq 7OPT/8$.

For clauses (a∨b∨c), (¬a∨b∨c), (a∨¬b∨c), (a∨b∨¬c), initial expectation is 7/2. Branch a=0 gives 13/4 and a=1 gives 15/4, so choose 1. Then b=0 gives 7/2 and b=1 gives 4, so choose 1. Either c works; choose 0 and all four clauses hold.

Recomputing m clauses at each of n variables takes O(nm) fixed-precision fraction operations. Denominators here are at most 8, with no hidden exponential precision.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Weighted vertex-cover LP and threshold rounding

With positive rational weights, minimize $\sum_v w_v x_v$, subject to $x_u + x_v \geq 1$ on every edge and integral x in {0,1}. Relax to 0≤xv≤1. Every integral cover remains feasible, so $LP^* \leq OPT$.

Find an optimal feasible LP solution x* and choose C={v:xv*≥1/2}. If both endpoints of an edge were omitted, both values would be below 1/2, violating the edge constraint. Thus C is a cover. For every vertex, its selection indicator is at most 2xv*. Multiply by nonnegative weights and sum:

$$w(C) \leq 2 \sum_v w_v x_v^* = 2LP^* \leq 2OPT.$$

Rational LP has polynomial-bit-complexity algorithms. Frequent practical success of simplex does not prove its polynomial worst case. The proof uses an actually feasible LP solution. Numerical tolerances need their own feasibility and cost analysis. The number 0.4999 is not exactly 1/2.

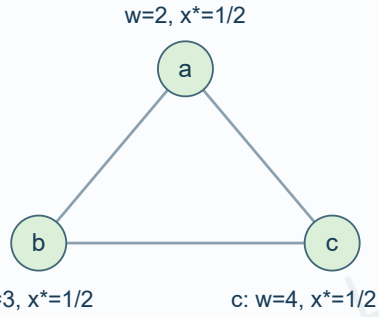
LP optimum differs from integer optimum

For a triangle with weights 2,3,4, the half-value solution costs 9/2. Multiply edge constraints ab,ac,bc by 1/2,3/2,5/2. Their left side becomes $2x_a + 3x_b + 4x_c$, with right side 9/2. Every LP solution therefore costs at least 9/2, proving the half solution optimal.

An integer cover needs at least two vertices. Their costs are 5,6,7, so OPT=5. Threshold rounding takes all three, giving ALG=9=2LP*≤2OPT=10. Its actual ratio is 9/5. The LP value 4.5 is not the original optimum.

LP optimum differs from integer optimum (continued)

Weighted triangle: LP*=9/2, integer OPT=5, rounded cost=9



Edge multipliers: ab 1/2, ac 3/2, bc 5/2

Sum: $2x_a+3x_b+4x_c \geq 9/2$

Half values attain 9/2, proving LP optimality

Integer cover a,b costs 2+3=5

Threshold ≥1/2 selects all three: cost 9

Green vertices all pass the threshold. LP and integer optima are different.

Randomized set-cover rounding

For N≥2, use the LP constraints $\sum_{S \ni e} x_S \geq 1, x_S \geq 0$, with nonnegative costs. Set integer $k = 3\lceil \log_2 N \rceil \geq 3 \ln N$, and select each set independently with $p_S = \min(1, kx_S)$. Rational x gives rational probabilities, implementable by uniform integer sampling over their denominators. No irrational logarithmic probability is required.

If some set containing e has p=1, e is certainly covered. Otherwise, its omission probability is

$$\prod_{S \ni e}(1 - p_S) \leq e^{-\sum p_S} \leq e^{-k} \leq N^{-3}.$$

A union bound gives probability at most $N^{-2} \leq 1/4$ of any uncovered element. Expected cost is at most kLP*. By Markov, cost exceeds 2kLP* with probability at most 1/2. Adding the two failure probabilities gives at most 3/4, so each trial simultaneously covers X and meets the cost bound with probability at least 1/4. The ratio 2k is O(log N).

Randomized set-cover rounding (continued)

Check actual coverage and cost, and independently retry failures. Expected trials are at most four. This is an expected polynomial algorithm with a verifiable success condition, not a guaranteed single successful trial. For N=0 return empty. For N=1 choose the cheapest set containing its element. If LP*=0, nonnegative cost and zero expected cost imply cost zero with certainty. The cost check then automatically passes; only coverage needs retries, without dividing by zero in Markov's inequality.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Optimization subset sum

Given positive integers a1,...,an and integer $t \geq 0$, maximize a subset sum at most t. Items are distinguished by indices, even if values repeat. Remove ai>t first. For n=0 or t=0 return the empty subset with value zero.

The exact list algorithm starts L0={0}. At step i, merge and deduplicate Li-1 and Li-1+ai, then remove values exceeding t. Both lists are sorted, so two-pointer merging is linear without re-sorting.

Every subset of the first i items either excludes i or includes it, corresponding to those two lists. Positivity means an oversized sum can never decrease later, so removal is safe. Induction retains every feasible exact sum. There can be 2^i distinct sums, for example powers of two with a sufficiently large target. Linear merging per round does not make the overall algorithm polynomial.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

TRIM retains smaller, actual sums

For a sorted list and δ>0:

```
R=[]
for y in L, in increasing order:
    if R is empty or y>(1+delta)*R[-1]: append y to R
return R
```

If y is omitted, the retained last value z satisfies $z \le y \le (1+\delta)z$. If retained, choose z=y. Every candidate therefore has a representative with $y/(1+\delta) \le z \le y$. Zero is kept, and the first positive value cannot be removed by zero.

Retained values are actual integer subset sums. TRIM does not divide or round their numerical values. Smaller representatives preserve capacity feasibility when later items are added. A larger representative can exceed the target even when the original y remains feasible.

Merge, trim, and remove oversized values

For $0 < \epsilon < 1, n \ge 1$, set $\delta = \epsilon/(2n)$.

```
L=[0]
for i=1..n:
    B=merge_unique(L, [z+a[i] for z in L])
    L=trim(B,delta)
    remove all values >t from L
return max(L)
```

Every retained value is an actual sum from the first i items, so the returned sum is feasible and at most t. To return its subset, retain immutable parent pointers and whether the current item was used. When equal values merge, choose any valid provenance. Backtracking takes at most n steps, with strictly decreasing item indices and no item reuse.

Value-only computation can roll its lists. Retaining reconstruction nodes for all rounds instead needs O(nK) nodes, where K bounds one list. It cannot still be claimed to use only O(K) space.

Induction includes the selected-item branch

For every exact feasible sum y≤t from the first i items, prove there is $z \in L_i$ with $y/(1+\delta)^i \le z \le y$. At i=0, only y=z=0 exists.

If item i is excluded, induction gives a prior representative u with $y/(1+\delta)^{i-1} \le u \le y$. Candidate B contains u. TRIM supplies z≥u/(1+δ), yielding the bound. Also z≤y≤t, so final removal does not discard it.

If item i is included, write y=y′+ai. Positivity ensures y′≤t. Induction gives u between $y'/(1+\delta)^{i-1}$ and y′. Candidate B contains u+ai, and

$$u + a_i \ge y'/(1+\delta)^{i-1} + a_i \ge (y' + a_i)/(1+\delta)^{i-1}.$$

TRIM loses at most one additional factor 1+δ. Its representative is at most u+ai≤y≤t, so survives capacity removal. The second inequality explicitly uses ai≥0. With negative items, both early removal and this proof need revision.

Accumulated error stays within epsilon

Apply the invariant to OPT. The maximum retained value is at least its representative, so $OPT/z_{max} \le (1+\delta)^n$ when OPT>0. For OPT=0, use the approximation inequality directly.

Since $\ln(1+x) \le x$,

$$n \ln(1 + \epsilon/(2n)) \le \epsilon/2.$$

For 0<ε<1, $\ln(1+\epsilon) = \int_0^\epsilon (1+x)^{-1} dx \ge \epsilon/(1+\epsilon) \ge \epsilon/2$. Therefore $(1+\epsilon/(2n))^n \le 1+\epsilon$, giving

$$z_{max} \ge OPT/(1+\epsilon) \ge (1-\epsilon)OPT.$$

The proof controls multiplication of per-round errors. Setting every round's δ equal to global ε would allow factor $(1+\epsilon)^n$ and would not establish the same global guarantee.

Four rounds with integer lists

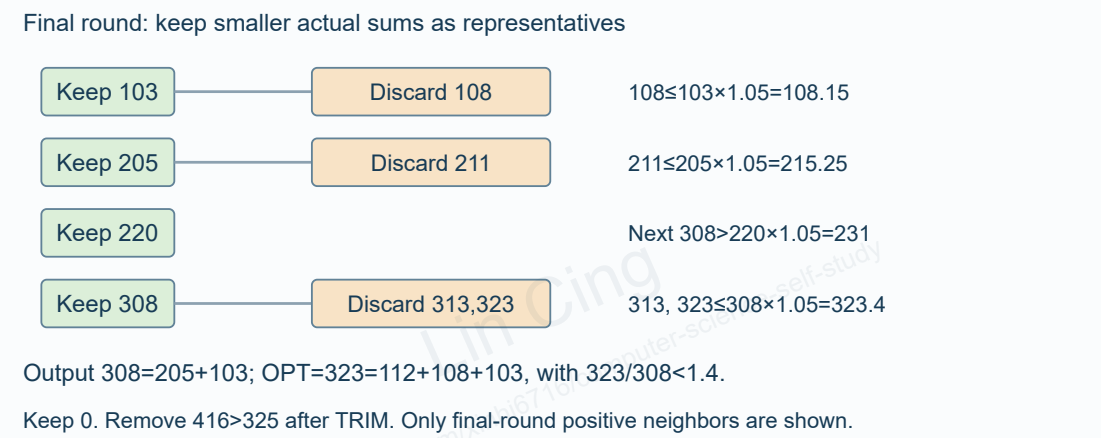
Use a=[112,108,205,103], t=325, ε=2/5, n=4, δ=1/20.

i	Merged unique B	After TRIM	After removing values above 325
1	0,112	0,112	0,112
2	0,108,112,220	0,108,220	0,108,220
3	0,108,205,220,313,425	Keep all	0,108,205,220,313
4	0,103,108,205,211,220,308,313,323,416	0,103,205,220,308,416	0,103,205,220,308

At round 2, 112≤108・1.05=113.4, so discard that candidate sum. At round 4, 108≤103・1.05=108.15, 211≤205・1.05=215.25, and both 313,323≤308・1.05=323.4. Each is represented by a smaller actual sum.

The output is 308=205+103. Exact OPT=323=112+108+103. The actual ratio is about 1.049≤1.4, and 308≤325. Discarding candidate sum 112 at one round does not delete that input item.

Four rounds with integer lists (continued)



List length, bit cost, and FPTAS

Adjacent positive trimmed values have ratio greater than 1+δ. The smallest is at least 1 and the largest at most t. There are at most $1 + \ln t / \ln(1+\delta)$ positive entries, plus zero. Since $\ln(1+\delta) \geq \delta/(1+\delta) \geq \delta/2$,

$$K = O(1 + n\log(t+1)/\epsilon).$$

Linear merging, trimming, and filtering over n rounds use $O(nK) = O(n + n^2\log(t+1)/\epsilon)$ arithmetic operations and comparisons. Values alone need O(K) workspace. The logarithm is input bit length, unlike the numerical t in exact DP's O(nt).

For exact rational $\epsilon = \alpha/\beta$, test trimming by $2n\beta y > (2n\beta + \alpha)last$, avoiding floating relative error. Reading input and removing ai>t costs O(N) bit work, with original bit length N. Later candidates are at most 2t. Intermediate bit lengths are polynomially controlled by log(t+1), log n, and the encoding of α,β. Including each large-integer multiplication still gives time polynomial in input length and 1/ε, completing the FPTAS proof.

Guarantees that must remain distinct

Method	Feasibility	Objective guarantee	Conditions
Matching endpoints for unweighted VC	Every run	ALG≤2OPT	Does not cover arbitrary weights
Double-tree TSP	Every run	ALG≤2OPT	Complete, nonnegative, symmetric metric
Weighted set-cover greedy	Every run	ALG≤Hd OPT	Choose cost per newly covered element
Random MAX-3-CNF	Every run returns an assignment	E[ALG]≥7OPT/8	Three distinct variables per clause
LP threshold weighted VC	Every run	ALG≤2OPT	Optimal feasible LP and nonnegative weights
Subset-sum TRIM	Every run	ALG≥OPT/(1+ε)	Nonnegative items and δ=ε/(2n)

Guarantees that must remain distinct (continued)

Expectation, deterministic inequalities, and probability-of-failure statements answer different questions. An approximation scheme must account for precision, list length, encoded input size, and reconstruction cost. Removing nearby values alone is not a running-time proof.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Source

Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*, 3rd ed., MIT Press, 2009, Chapter 35.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

References and attribution

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study